AI651 — Deep Learning for Space, Time and Graphs · Fall 2026 · LUMS

Assignment 2 Report

Muhammad Awais Ismail · Roll number 24030007

Code repository: https://github.com/Awais65/DL4STG-PA2

Hardware for all timings: NVIDIA GeForce RTX 4070 Laptop GPU (8 GB), Intel Core i9-14900HX, 32 GB RAM; Windows 11, PyTorch 2.6.0, CUDA 12.4.

Question 1 — River Forecasting Proposal

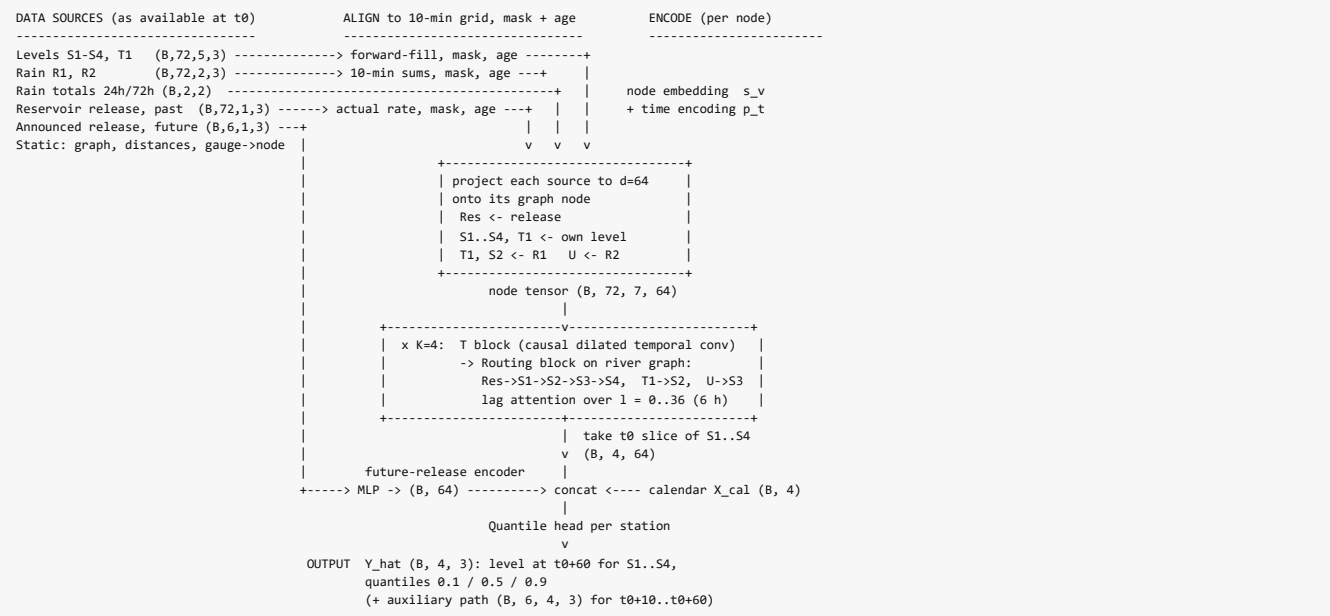
Service: every 10 minutes, at issuance time t_0, forecast the water level one hour ahead (t_0 + 60 min) at the four main-river stations S1–S4.

The design in one sentence: a **directed river graph** whose nodes are the reservoir, the stations and a latent node for the unmonitored tributary. Each node first runs its own causal **temporal encoder**. **Routing blocks** then pass information downstream along the river edges, using **flow-dependent travel-time attention** over past lags. Finally a **quantile head** predicts each station's level change from now to t_0 + 60 min.

---

1. Architecture diagram

In tensor shapes, B = batch, H = 72 past steps of 10 min and d = 64 features.



The graph has seven nodes: Res, S1, T1, S2, U (the unmonitored tributary, a latent node with no measurements), S3 and S4. Its edges are directed downstream.

---

2. Assumptions

These are assumptions, not facts. The supplied facts are the station layout, the 10-minute cadence, which tributary is monitored (T1, joining between S1 and S2) and which is not (joining between S2 and S3), the available sources, and that future rainfall and future water levels are unknown at issuance.

- **A1 — Common clock and datum.** All timestamps share one timezone (UTC) and the stations report levels on a common, stable datum. *Why reasonable:* this is standard for an authority-run network. *Effect:* records can be aligned on one 10-minute grid, and a level change at one station is comparable with one at another.
- **A2 — Record length.** At least 2–3 years of history, including several high-flow events. *Why:* the authority runs flood preparedness, so archives should exist. *Effect:* enough data to learn rainfall response and flow-dependent travel times. With a shorter record, the design leans more on the physics-based prior (A5).
- **A3 — Reporting delays.** Levels and rainfall normally arrive within 10–20 minutes of measurement, with occasional outages of hours. Each record carries both its measurement time and its arrival time. *Effect:* inputs are built "as of" t_0 from arrival times (§5), and each source gets an age feature.

- **A4 — Response times.** Travel time from the reservoir to S4 and the rainfall-to-river response are both under about 12 hours. Antecedent wetness is summarised by 24-hour and 72-hour rainfall totals. *Effect:* a history window of H = 72 steps (12 h) is long enough, and each routing edge needs lags of at most 6 hours.
- **A5 — Fixed flow direction, monotone response.** Water flows only downstream, with no significant backwater from below S4. More release or more rainfall never lowers downstream levels. *Effect:* the graph's edges are directed, and the lag-attention prior is built from the distances along the river. Monotonicity is not built into the network; it is used as a check in verification (a larger release must not lower the forecast).
- **A6 — Gauge-to-catchment mapping.** R1 mostly represents the catchment draining between S1 and S2, including T1's basin. R2 represents the unmonitored tributary's catchment. *Why:* the gauge locations are supplied, but which areas drain where is assumed. *Effect:* R1 feeds nodes T1 and S2, and R2 feeds the latent node U. If the mapping is wrong, a learned gauge-to-node projection can partly correct it.
- **A7 — Release schedules.** Schedules are announced before issuance and cover at least the next 60 minutes. Actual releases are recorded within the reporting delay. Past announcements are archived *as announced*, not overwritten later by the actual values.
- **A8 — Stable level–flow relation.** Each station's rating curve is stable: no ice, and no large changes to the channel within the record. *Effect:* the model can work in level directly, without converting to discharge.
- **A9 — Measurement quality.** Sensor noise is a few centimetres, smaller than forecast-relevant changes, and occasional spikes are filtered out by simple quality-control rules.
- **A10 — Stationarity.** Apart from seasonal patterns, the system's behaviour doesn't change over the record. Reservoir operating rules may change; this is handled by retraining (§6).

---

3. Forecasting task and data specification

- **Issuance time:** t_0 = every 10 minutes, on the 10-minute grid (:00, :10, …, UTC). An input counts only if it **arrived** by t_0.
- **History window:** H = 72 steps, covering $t_0 - 710$ min up to t_0 (12 h, A4). This is supplemented by 24-hour and 72-hour rainfall totals.
- **Horizon:** the level at $t_0 + 60$ min at S1–S4. An auxiliary output predicts the full path $t_0 + 10 \dots t_0 + 60$, which is used only as a training aid.

Name	Type	Shape	Axes (in order)	Time coverage
`X_level`	input	(B, 72, 5, 3)	batch; time step; station [S1, S2, S3, S4, T1]; [level (forward-filled, normalised), observed mask, age in min]	$t_0-710 \dots t_0$
`X_rain`	input	(B, 72, 2, 3)	batch; time; gauge [R1, R2]; [rain depth per 10 min, mask, age]	$t_0-710 \dots t_0$
`X_rainctx`	input	(B, 2, 2)	batch; gauge; [24 h total, 72 h total]	to t_0
`X_rel_past`	input	(B, 72, 1, 3)	batch; time; reservoir; [actual release rate, mask, age]	$t_0-710 \dots t_0$
`X_rel_future`	input	(B, 6, 1, 3)	batch; future step; reservoir; [scheduled rate, announced flag, min since announcement]	$t_0+10 \dots t_0+60$
`X_cal`	input	(B, 4)	batch; [sin, cos time of day; sin, cos day of year]	t_0
`G`	static	7 nodes, 6 edges + distances; gauge→node map	—	fixed
`Y_hat`	output	(B, 4, 3)	batch; station [S1…S4]; quantile [0.1, 0.5, 0.9]	t_0+60
`Y_aux`	output (training only)	(B, 6, 4, 3)	batch; horizon; station; quantile	$t_0+10 \dots t_0+60$

Alignment.

- **Levels:** placed on the 10-minute grid by their measurement time. A missing slot is forward-filled, with mask = 0 and the age counting up.
- **Rainfall:** summed over each 10-minute interval ending at a grid time.
- **Releases:** treated as a step function sampled at grid times. Past values come from actual records, future values from the announced schedule.

Each source is mapped onto its graph node with its own learned linear projection to d = 64. The reservoir release goes to Res, each station's level to its own node, rainfall to the nodes given by A6, and nothing to U except R2. This turns sources with different numbers of locations and features into one node tensor of shape (B, 72, 7, 64).

Available at issuance: past levels, rainfall and actual releases that arrived by t_0, plus release schedules announced by t_0. **Not available:** future rainfall, future levels, any record that arrives after t_0, and any measurement of the unmonitored tributary.

---

4. Model computations and design justification

Notation. The graph has nodes V = {Res, S1, T1, S2, U, S3, S4} and directed edges E = {Res→S1, S1→S2, T1→S2, S2→S3, U→S3, S3→S4}. Time steps $\tau = 1 \dots H$, with $\tau = H$ corresponding to t_0. $h_{v,\tau} \in \mathbb{R}^d$ is node v's representation at time τ.

(1) Input embedding. For each node v,

$h^{(0)}_{v,\tau} = W_{src(v)}\,[x_{v,\tau};\, m_{v,\tau};\, a_{v,\tau}] + \Sigma_g\, B_{gv}\, W_{rain}\,[r_{g,\tau};\, m_{g,\tau};\, a_{g,\tau};\, c_g] + s_v + p_\tau$,

where s_v is a learned node embedding, p_τ is a time encoding, $B_{gv} \in \{0,1\}$ is the gauge-to-node map from A6, and c_g holds gauge g's 24 h and 72 h rainfall totals from `X_rainctx` (the same at every τ). The latent node U has no level term, so its embedding comes only from s_U, p_τ and R2's rainfall.

(2) T block. This is a causal dilated temporal convolution over each node's own history, with a residual connection: h ← h + TCN(h). It uses kernel 3 with dilations 1, 2, 4, 8 and 16, giving a temporal receptive field of 63 steps per block; the K = 4 stacked T blocks together cover far more than the 72-step window. Being causal, it never looks past t_0.

(3) Routing block (flow-dependent lag attention). For each edge u→v, the message is a weighted sum over the upstream node's recent past:

$z_{u\to v,\tau} = \Sigma_{\ell=0}^{L}\, \alpha_{uv,\ell}(\tau) \cdot W_e\, h_{u,\tau-\ell}$, L = 36 steps (6 h),

$\alpha_{uv,\cdot}(\tau) = \mathrm{softmax}_\ell(\, q(h_{v,\tau},\, h_{u,\tau})^{\top} k_\ell + b_{uv,\ell}\,)$,

and the node is updated with $h_{v,\tau} \leftarrow h_{v,\tau} + \mathrm{MLP}([h_{v,\tau};\, \Sigma_{u\in in(v)}\, z_{u\to v,\tau}])$. Here q is a small MLP that forms a query from the current states of both nodes, and k_ℓ is a learned embedding of lag ℓ. The bias $b_{uv,\ell}$ is initialised as a Gaussian centred on the expected travel lag, the reach's length divided by a typical wave speed. This is the physical prior. Because the attention query depends on the current state, a high-flow state can move the attention to shorter lags, matching flood waves that travel faster. The mapping from flow state to lag is learned.

(4) Depth and reach. The longest path in the graph is Res→S1→S2→S3→S4, which is 4 hops. So K = 4 routing blocks, each preceded by a T block, are needed for a release or upstream event to influence S4 within one forward pass. Temporal reach covers the 72-step window, and each hop adds up to 6 hours of lag.

(5) Forecast head. At issuance, for each main station s and quantile q ∈ {0.1, 0.5, 0.9}:

$\hat{y}_{s,0.5} = y_{s,t_0} + g_s([h^{(\kappa)}_{s,H};\, c_{rel};\, c_{cal}])$, $\hat{y}_{s,0.1} = \hat{y}_{s,0.5} - \mathrm{softplus}(\cdot)$, $\hat{y}_{s,0.9} = \hat{y}_{s,0.5} + \mathrm{softplus}(\cdot)$,

where c_{rel} = MLP(`X_rel_future`) encodes the announced releases and c_{cal} encodes the calendar features. The model predicts the *change* from the last observed level, a residual on persistence. The softplus offsets keep the quantiles in order.

(6) Training objective. The loss is a pinball (quantile) loss, masked where the true value is missing:

$\mathcal{L} = \Sigma_s\, \Sigma_q\, w_s(y) \cdot m_s \cdot \rho_q(y_{s,t_0+60} - \hat{y}_{s,q}) + \lambda \cdot \mathcal{L}_{aux}(t_0{+}10 \ldots t_0{+}50)$, with $\rho_q(e) = \max(q{\cdot}e,\, (q-1){\cdot}e)$.

Here λ (e.g. 0.1) weights the auxiliary path loss. The weight $w_s(y)$ up-weights levels above the authority's alert thresholds, so the rare floods that matter most aren't swamped by normal-flow periods. The training data mimics real operations: sensors and intervals are dropped at random, and arrival delays are simulated (§5).

Justification and influence paths.

- **Why a graph, not a grid or full attention.** The river's connectivity is known, so the model's paths follow it: each station receives information only from what is physically upstream. Lag attention costs O(|E|·H·L·d) per block. That is small, and it avoids the cost of full space–time attention, which grows with the square of the number of tokens. A 4-hop graph covers the whole river, whereas a local convolution would need far more depth to reach that far.
- **Physical versus learned.** Physical assumptions: flow direction, connectivity, causality, the lag prior from distances, and predicting the change from persistence. Learned: the travel-time attention weights, the rainfall-to-level response, the latent tributary's inflow, how far actual releases deviate from schedules, and the uncertainty.
- **What drives each forecast.**
  - **S1:** reservoir releases (past actual values through Res→S1, plus announced future values through c_{rel}) and its own recent level. No rain gauge covers the catchment above S1 (A6).
  - **S2:** S1 routed downstream, T1 routed downstream (the monitored tributary), and R1 rainfall.
  - **S3:** S2 routed downstream, plus the latent U node, which is driven by R2 rainfall. The model learns how much S3 rises beyond the routed S2 signal.
  - **S4:** everything above, through four hops.

---

5. Operation and exceptional conditions

(i) Missing observations. Each missing slot is forward-filled, with mask = 0 and the age feature counting up. Because training drops sensors and stretches of time at random, the model learns to rely more on upstream stations and rainfall when a station's own record is stale.

- **If a station's own level is missing at t_0:** the forecast is still issued from its last value and the upstream information. It is flagged, and its quantile interval widens with the age feature.
- **If a key input is missing for more than about 6 hours:** for example, S1 offline while a release is under way. The system falls back to a simple routing-plus-persistence model and labels the forecast as degraded.

(ii) Delayed observations. Inputs are taken "as of" t_0 from arrival times (A3). A record that arrives late is never used for a forecast that was issued before it arrived, so training and operation see exactly the same information. The age features tell the model how fresh each source is, and training simulates realistic delays. Late records enter the next issuance 10 minutes later, so the forecast is revised then. Issued forecasts are archived unchanged, for verification.

(iii) Announced future releases. The schedule for t_0+10 … t_0+60 enters through `X_rel_future` and c_{rel}, with an "announced" flag and the time since announcement. Only schedules announced by t_0 are used. Releases that started earlier but are still travelling downstream are already in `X_rel_past` and reach later stations through the routing blocks.

(iv) Scheduled versus actual releases.

- **Past versus future:** past inputs always use the **actual** recorded release, and only the future uses the schedule.
- **Training:** training uses historical schedules *as they were announced* (A7). The model therefore learns how far actual releases typically deviate from the schedule and builds that into its uncertainty.
- **Detecting a mismatch:** the last actual release minus the last scheduled release is added as an extra input to c_{rel}. It exposes a current mismatch, which widens the S1 interval.
- **Correction:** once the actual release is recorded, the next issuance uses it.

Feasibility every 10 minutes. Each forecast processes one window of 7 nodes × 72 steps × 64 features through 4 T blocks and 4 routing blocks over 6 edges. That is about 400,000 parameters and on the order of 10^8 multiply-adds per forecast, which takes tens of milliseconds on a CPU. The real work each cycle is in data ingestion and quality control, and both are routine. There are two levels of fallback: the routing-plus-persistence model, and persistence alone.

---

6. Uncertainty, limitations and additional information

Each limitation below gives its impact on the forecasts and the response.

- **The unmonitored tributary.** Its inflow between S2 and S3 is never measured. It is modelled only through the latent node U, driven by R2 rainfall, and learned from how much S3 rises beyond the routed S2 signal.
  - *Impact:* larger errors at S3 and S4 during local storms over that catchment, especially storms R2 doesn't capture.
  - *Response:* the quantile intervals widen with R2 rainfall and with recent unexplained rises at S3, and verification reports S3 and S4 performance separately.
  - *Unresolved:* storms that miss both gauges.
- **Changing travel times.** Travel times change with river conditions, which the flow-dependent lag attention handles.
  - *Impact:* in extreme floods outside the training range, the attention weights have to extrapolate and the predicted timing may be off.
  - *Response:* flood events are up-weighted in training, the lag prior is physics-based, and the intervals widen when levels go beyond the training range.
- **Sparse rainfall and unknown future rain.** Two gauges can miss convective cells, and future rainfall isn't available.
  - *Impact:* errors for rain that starts after t_0 and reaches a station within the hour. This matters mostly for fast local runoff near S2 and S3.
  - *Response:* this is accepted for a one-hour horizon, where most of the signal is already in the river and is reflected in wider intervals.
- **Reservoir operations.** Unannounced or changed releases are unknowable in advance. They are partly covered by learned schedule-deviation uncertainty (§5 iv), and the forecast corrects within one or two issuances.
- **Non-stationarity and sensor drift.** Channel changes, a new rating curve, a datum change or new operating rules all break A8 and A10. *Response:* retrain on a rolling schedule, monitor errors continuously, and alert operators to data-quality problems.
- **Unspecified data conditions.** The record length, reporting delays and quality flags aren't known (A2, A3, A9). If they turn out worse than assumed, the history window, the fallback thresholds and the training-time simulation all have to be retuned.

Additional information to request. The system remains usable without any of these.

Request	How it helps	Usable without?
A temporary or permanent gauge on the unmonitored tributary, or its catchment area and land use	Turns U from a latent node into an observed one, or gives a physical prior for its inflow, cutting S3 and S4 errors	Yes, through the latent node and R2
Rating curves and channel cross-sections at S1–S4	Converts levels to discharge, gives physically based travel times, and helps extreme floods	Yes, the model learns in level space
Archive of past release schedules *as announced*, plus operating rules	Lets the model learn how schedules deviate from actual releases (§5 iv)	Partly. Without it, the deviation uncertainty is set by hand
Arrival-time logs and quality flags	Lets the "as-of" simulation match real delays and outages	Yes, with assumed delays (A3)
Rainfall radar or short-range rain nowcasts	Fills gaps between the two gauges and adds rain expected after t_0	Yes
Temperature and snow data (for the mountain catchment)	Captures seasonal snowmelt, which the rainfall inputs miss	Yes, seasonality is partly captured by calendar features
Historical flood reports and alert thresholds	Sets the loss weights w_s, and serves as verification cases	Yes, with thresholds estimated from the data

The model treats uncertainty through its quantile outputs, trained with the pinball loss. Their calibration is checked on held-out years by asking whether 80% of outcomes fall inside the 0.1–0.9 band, both overall and for high-flow periods.

Question 2 — Motion or Memory?

All numbers use the 1,000 test videos (9000–9999) at t = 1 except Table 1 (synthetic tensors), Figure 1 (training videos), parameter counts and timings. In every table, learned models are reported as mean (seed 0, seed 1); figures, GIFs and histograms use seed 0. Errors are given to three significant figures.

1. Setup

Table 1. Sanity checks of the transforms and speed windows.

Check	Expected	Yours
(a) max|M(M(x)) - x|	0	0 (torch.equal: True)
(b) max|R^4(x) - x|	0	0 (torch.equal: True)
(c) max|M R M(x) - R^{-1}(x)|	0	0 (torch.equal: True)
(d) bright pixel of M(p)	(7, 10, 43)	(7, 10, 43)
(e) bright pixel of R(p)	(7, 43, 10)	(7, 43, 10)
(f) 1× input / target frames	1, 2, 3, 4, 5 / 6, 7, 8, 9, 10	1, 2, 3, 4, 5 / 6, 7, 8, 9, 10
(g) 2× input / target frames	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19
(h) 3× input / target frames	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19

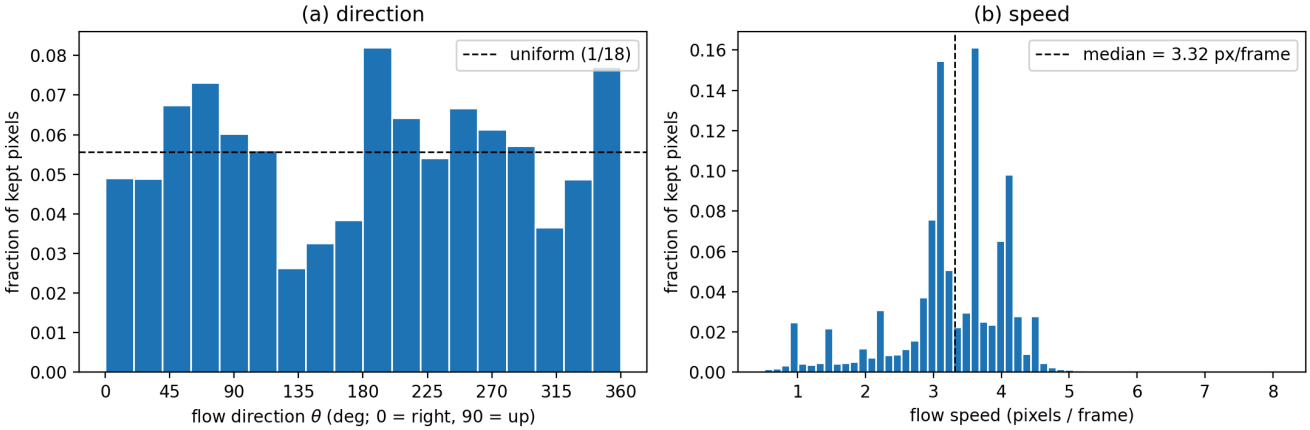


Figure 1. Farneback optical flow on the first 50 training videos (950 consecutive-frame pairs), pooled over the 103,050 kept pixels (intensity > 0.2 in both frames and flow magnitude > 0.5 px). (a) Direction histogram in 20° bins, 0° rightward and 90° upward, with the uniform level 1/18 marked. (b) Speed histogram; median 3.32 px/frame (vertical line). Both as fractions of kept pixels.

Q2.1

Figure 1(a) shows motion in every direction. Each 20° bin holds between 0.47× and 1.48× the uniform share of 1/18, and no bin is empty. Mirroring every video maps a direction θ to 180° − θ, and rotating by 90° maps it to θ + 90°. Either way the histogram is reshuffled, but every new direction already occurs in training. The mirrored histogram differs from the original by 13% of its mass (total variation distance) and the rotated one by 20%. Most of that difference is likely sampling noise, since the 50 videos contain only about 100 digit trajectories. So mirrored and rotated trajectories should already look familiar as motion. Figure 1(b) gives a typical speed of 3.3 px/frame (median), with 80% of kept pixels between 2.1 and 4.1 px/frame. At 2× a digit moves about 6.6 px/frame and at 3× about 10 px/frame. Only 21 of the 103,050 kept pixels (0.02%) move faster than 6.6 px/frame, and 99.9% move slower than 5.7 px/frame, so 2× and 3× motion is essentially absent from the data. Optical flow is unreliable where digits overlap: two digits moving differently in one region produce mixed or averaged vectors, and the brightness of a digit that's being occluded changes. That noise mostly affects a small fraction of pixels and is not tied to any one direction, so it could blur the histograms but not create or remove whole directions. The few pixels above 6.6 px/frame are more likely such flow errors than real fast motion. My conclusion holds.

Q2.2

- Mirror changes both: it maps direction θ to 180° − θ, and it produces mirrored digits (a backwards 2, 3, 4 or 7) that never occur in training.
- Rotation by 90° also changes both: directions shift by 90°, and the digits appear sideways.
- Speed-up changes only the motion: it skips frames, so the displacement per frame doubles or triples, but each frame shows ordinary upright digits.

By Q2.1, mirror and rotation keep the motion statistics familiar while making the appearance unfamiliar. Speed-up does the opposite: familiar appearance, unfamiliar and larger motion. All three keep the physics valid (constant velocity, elastic bounces) and change no other property of the data. A failure under mirror or rotation therefore points at appearance. A failure under speed-up points at motion, either unseen speeds or the architecture's reach. That makes them a controlled experiment: each transformation isolates one cause.

2. Baseline: ConvLSTM

Table 2. Per-horizon test MSE and $E^{(5)}$ at 1×. Learned models: mean (seed 0, seed 1); Blank and Copy-last are deterministic.

Model	MSE_1	MSE_2	MSE_3	MSE_4	MSE_5	$E^{(5)}$	Params
A	1.66e-2 (1.71e-2, 1.61e-2)	2.01e-2 (2.06e-2, 1.95e-2)	2.24e-2 (2.29e-2, 2.19e-2)	2.42e-2 (2.46e-2, 2.37e-2)	2.58e-2 (2.63e-2, 2.54e-2)	2.18e-2 (2.23e-2, 2.13e-2)	480,784
Blank	4.26e-2	4.26e-2	4.26e-2	4.25e-2	4.25e-2	4.26e-2	0
Copy-last	4.92e-2	6.04e-2	6.69e-2	7.08e-2	7.32e-2	6.41e-2	0

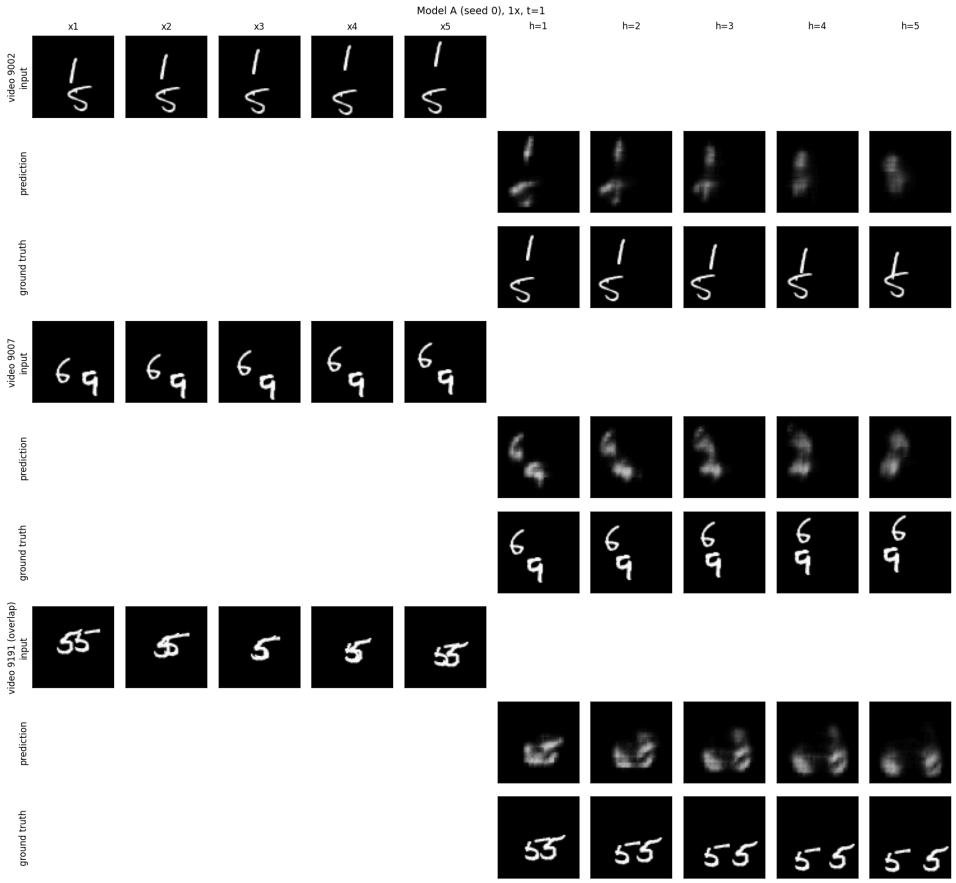


Figure 2. Seed-0 model A at 1×: inputs, predictions and ground truth for test videos 9002, 9007 and 9191 (digits overlapping).

Q2.3

Reach is measured centre-to-centre in original pixels. On the folded 16×16 grid, one cell is 4 px. Only the 3×3 convolutions inside the two ConvLSTM layers move information. With stride 1 and no dilation, each such convolution moves it at most one cell, which is 4 px. Pixel-unshuffle, pixel-shuffle, the 1×1 output convolution and the elementwise gate and cell updates move nothing. Within one step τ, the possible paths to the prediction $\hat{x}_{\tau+1}$ are:

- (i) from the input u_τ through layer 1's convolution (one cell), then through layer 2's convolution (one cell), to the output;
- (ii) from the previous state $h^{(1)}{}_{\tau-1}$, which enters the same layer-1 convolution, through the same two convolutions;
- (iii) from $h^{(2)}{}_{\tau-1}$ through layer 2's convolution only (one cell).

The cell states $c^{(\ell)}$ are updated elementwise, so they add no distance. Paths (i) and (ii) are the longest, at two cells, so model A's reach is 8 px per time step. That is above the typical training speed of 3.3 px/frame, close to 2× (about 6.6 px/frame) and below 3× (about 10 px/frame).

Q2.4

Model A beats both reference predictors at every horizon. Its $E^{(5)}$ is 2.18e-2, against 4.26e-2 for Blank and 6.41e-2 for Copy-last (Table 2), so it has clearly learned more than "predict nothing" or "nothing moves". Its error grows steadily with horizon, from MSE_1 = 1.66e-2 to

MSE_5 = 2.58e-2 (+55%). Blank's error is flat at about 4.26e-2, because it ignores the input. Copy-last is worse than Blank at every horizon and grows from 4.92e-2 to 7.32e-2. A copied digit sits in the wrong place, so it is penalised twice: once for the missing digit at its true position and once for the bright digit where nothing is. Figure 2 shows why model A's error grows. Its predictions put the digits in roughly the right place, but they become blurrier and dimmer with each step. Under MSE, an uncertain model hedges by spreading intensity over the positions it thinks are plausible, and the uncertainty compounds because each prediction is fed back as the next input. When digits overlap (video 9191), the merged input gives the model no clear separation. Its first predictions are a single blurred blob; at later horizons that blob splits into two blobs roughly where the digits really go, but neither is recognisable as a "5".

Q2.5

Prediction committed before the baseline tests (ƒf9a9db, 29 Oct 2026 11:29), reproduced unchanged:

Mirror. I expect model A to mostly pass: C_M should be small relative to $E^{(5)}$, and E_M close to $E^{(5)}$. Figure 1(a) shows training motion in every direction (no empty bins; roughly 0.5–1.5× the uniform level), so mirrored trajectories are familiar, and because convolutions apply the same local rule everywhere, the motion rule should transfer. The main risk is appearance: mirrored 2, 3, 4 and 7 never appear in training. Any C_M should therefore come from blurred or distorted digits, not wrong motion.

Rotation. I expect rotation to be harder: C_R and E_R larger than C_M and E_M. The rotated directions are still familiar, but a 90°-rotated digit is further from the training data than a mirrored one: sideways digits never occur, and some become ambiguous (e.g. a sideways 1 resembles a dash). If the motion rule depends on recognising digit shape, rotation should expose this more than mirroring.

Speed. Figure 1(b) gives a median speed of about 3.3 px/frame, so 2× ≈ 6.6 and 3× ≈ 10 px/frame. I estimate model A's reach at 8 px per step (Q2.3). 2× is just within reach, so predictions should degrade (as 2× motion is unseen in training) but stay recognisable. 3× exceeds the reach, so I expect breakdown: digits that stall, smear or vanish, with $E^{(2)}$ approaching or exceeding Copy-last. I predict breakdown at 3×, with clear degradation already at 2×.

Outcome (written afterwards): the mirror and rotation parts held (Table 3). The speed part was too optimistic: model A already breaks down at 2× (Table 4; see Q2.9).

3. Testing the Baseline

Table 3. Accuracy and mirror/rotation consistency at 1×. Learned models: mean (seed 0, seed 1); Blank and Copy-last are deterministic.

Model	$E^{(5)}$	C_M	E_M	C_R	E_R	Params
A	2.18e-2 (2.23e-2, 2.13e-2)	1.86e-3 (1.73e-3, 1.99e-3)	2.25e-2 (2.29e-2, 2.20e-2)	3.04e-3 (2.66e-3, 3.43e-3)	2.35e-2 (2.38e-2, 2.32e-2)	480,784
Blank	4.26e-2	0	4.26e-2	0	4.26e-2	0
Copy-last	6.41e-2	0	6.41e-2	0	6.41e-2	0

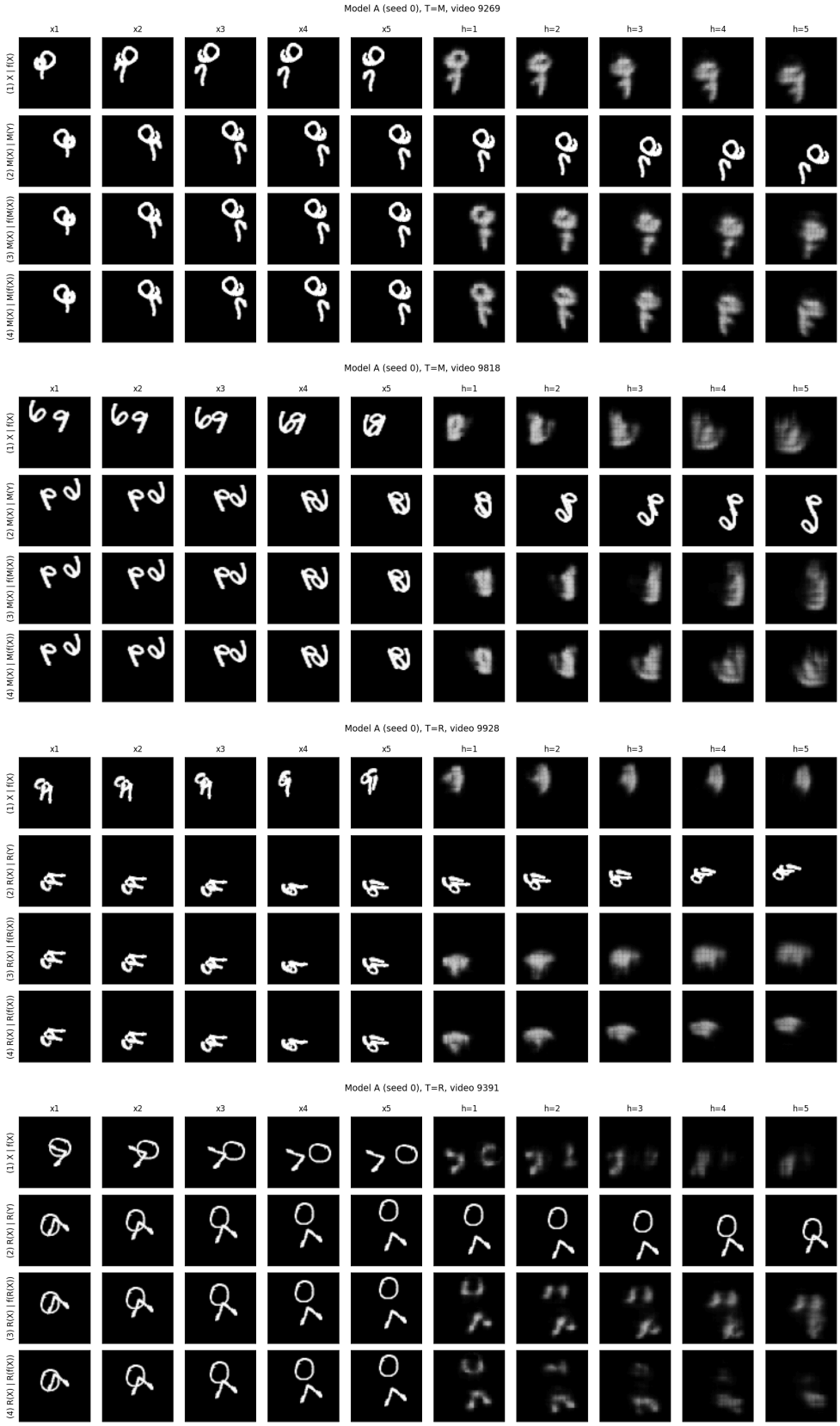


Figure 3. Seed-0 model A on the two test videos with the largest per-video C_M (9269, 9818; T = M) and the two with the largest per-video C_R (9928, 9391; T = R). Rows: (1) X | f(X), (2) T(X) | T(Y), (3) T(X) | f(T(X)), (4) T(X) | T(f(X)); columns 1–5 are the inputs, 6–10 the five forecast horizons. GIFs: `figures/gifs/fig3_M1_v9269.gif`, `figures/gifs/fig3_M2_v9818.gif`, `figures/gifs/fig3_R1_v9928.gif`, `figures/gifs/fig3_R2_v9391.gif`.

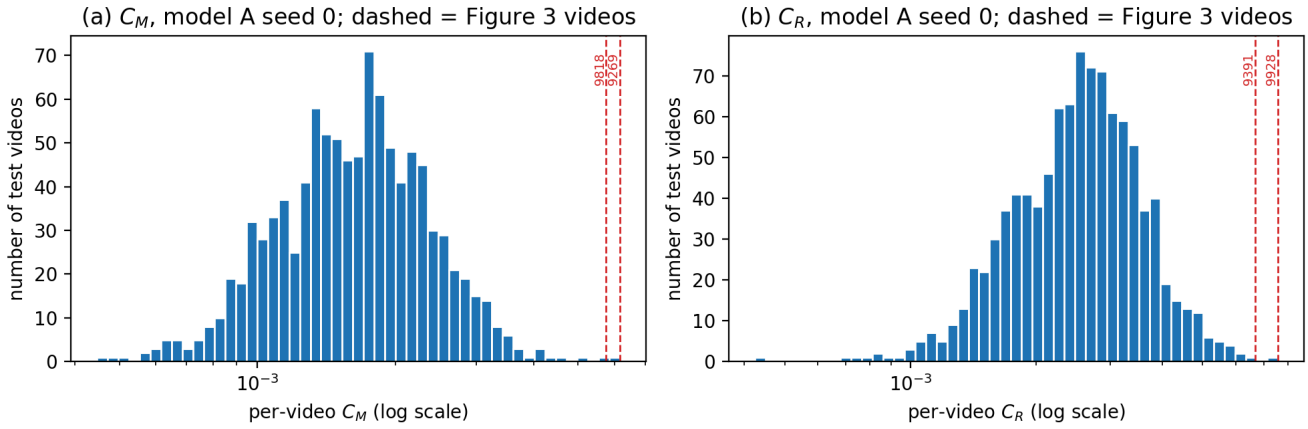


Figure 4. Histograms of per-video C_M (a) and C_R (b) for seed-0 model A over the 1,000 test videos (log scale); the Figure 3 videos are marked with dashed lines.

Q2.6

Blank and Copy-last commute with mirror and rotation by construction. Blank outputs zeros, and a transformed zero frame is still zero. Copy-last repeats the last input frame, and transforming the input and then copying gives the same result as copying and then transforming. So $C_M = C_R = 0$ exactly. Yet they are the two worst predictors: $E^{(5)}$ is 4.26e-2 for Blank and 6.41e-2 for Copy-last, against 2.18e-2 for model A (Table 3). C_T only asks whether the model treats a transformed clip the same way it treats the original. It says nothing about whether either prediction is right, so a trivial or badly wrong model can be perfectly consistent. Blurry predictions also lower C_T, because hedged outputs are similar to each other whatever the input. E_T supplies what C_T misses: accuracy on the transformed clips. A model has learned a rule that carries over only if C_T is small and E_T is close to its untransformed $E^{(5)}$. For model A, C_M = 1.86e-3 is small next to E_M = 2.25e-2, and E_M is only 3% above $E^{(5)}$.

Q2.7

Video 9928 under rotation (Figure 3, T = R). The video shows two overlapping 9s; they overlap in all 20 frames.

- Row 2, the true rotated future: the digits move straight up at about 3 px per frame. Their centre of brightness rises from row 43 to row 32 over the five horizons and stays near column 17.
- Row 4, R(f(X)), the model's prediction on the original clip, rotated afterwards: this follows the truth closely. It rises about 3 px per frame, from row 42 to row 30, drifts only about 1 px sideways, and stays fairly compact: the brightest pixel falls from 0.78 to 0.67.
- Row 3, f(R(X)), the model's prediction on the rotated clip: this tells a different story.
  - The upward motion slows to about 2 px per frame, ending near row 36 instead of 32.
  - The digits drift about 4 px to the right, a direction the true digits never move.
  - The digits smear. The bright area grows from 325 to 434 pixels and the brightest pixel falls from 0.80 to 0.58, so by h = 5 the two 9s are an unrecognisable blob.

The per-horizon consistency gap grows from 2.1e-3 at h = 1 to 1.2e-2 at h = 5. So on the rotated clip the model loses the digits' speed and direction, and the error compounds as its own blurred predictions are fed back.

Q2.8

Yes. In Table 3, C_R is 3.04e-3 against 1.86e-3 for C_M, about 1.6× higher, and C_R is higher for both seeds. E_R is 8% above $E^{(5)}$, while E_M is only 3% above. Figure 4 shows this isn't driven by a few outliers. The whole per-video C_R histogram sits to the right of C_M. C_R exceeds C_M in 89% of test videos, with medians of 2.58e-3 and 1.66e-3. By Q2.1, the difference is unlikely to come from the motion. After either transformation every direction still occurs in the training data. The rotated direction histogram differs from the original by 20% against 13% for the mirrored one, and most of that is sampling noise from about 100 trajectories. By Q2.2, the appearance does differ. A mirrored 0, 1 or 8 looks almost unchanged, and other mirrored digits keep their upright stroke structure. A 90° rotation turns every digit sideways, which is further from anything in training. So the extra difficulty most likely comes from appearance: the motion rule model A learned appears tied to upright digit shapes. Model A alone can't rule out a small motion contribution. Comparing it with model B, which saw every symmetry during training, tests this (Q2.13).

Table 4. Model A at 1×, 2× and 3×, with Copy-last for comparison. Learned models: mean (seed 0, seed 1); Blank and Copy-last are deterministic.

Model	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	$E^{(5)}$ 1×	$E^{(5)}$ 2×	Params
A	1.84e-2 (1.89e-2, 1.78e-2)	4.45e-2 (4.33e-2, 4.57e-2)	5.14e-2 (5.05e-2, 5.22e-2)	2.18e-2 (2.23e-2, 2.13e-2)	4.89e-2 (4.73e-2, 5.05e-2)	480,784
Copy-last	5.48e-2	6.61e-2	7.03e-2	6.41e-2	7.19e-2	0

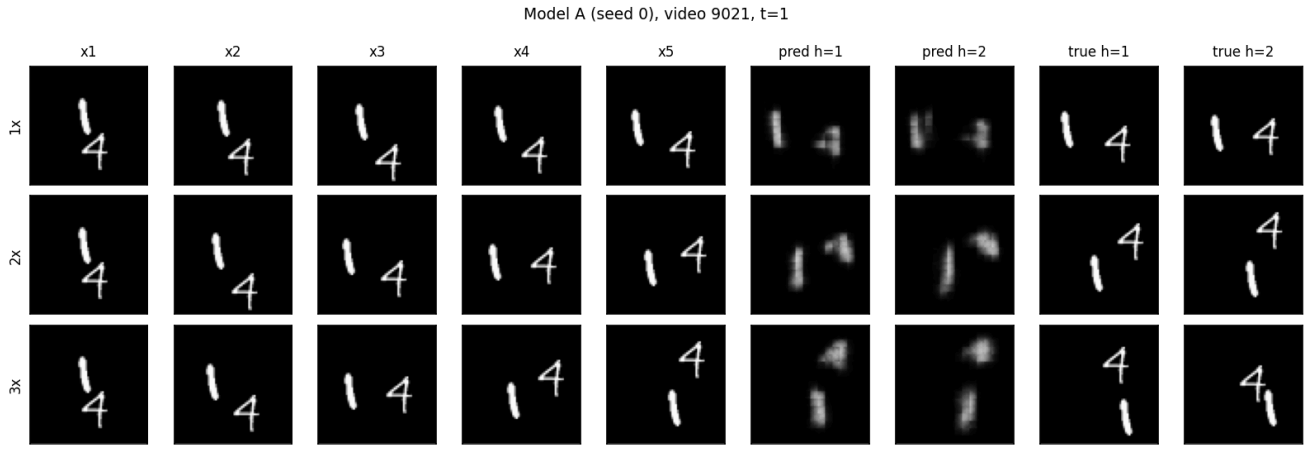


Figure 5. Seed-0 model A on test video 9021 (digits never overlap in the displayed true frames): the five input frames and the first two predicted and true future frames at 1×, 2× and 3×.

Q2.9

Model A falls apart at 2×. Its $E^{(2)}$ rises from 1.84e-2 at 1× to 4.45e-2 at 2×, which is 2.4 times higher and already worse than predicting a blank frame. Blank's $E^{(2)}$ is about 4.26e-2 at every speed. At 3× it reaches 5.14e-2. It stays below Copy-last (6.61e-2 and 7.03e-2), so it still extracts something from the input (Table 4). This only partly agrees with my Q2.3 reach of 8 px per step. 3× (about 10 px/frame) is beyond that reach, so failure there is expected. 2× (about 6.6 px/frame) is within it, yet the model still fails. What my estimate missed is that reach is only an upper bound on how far information can move in one step. It says nothing about what the model learned to do. Every training speed was below about 5.7 px/frame, so the learned update moves digits only about 3 px per step. Figure 5 (video 9021) shows this:

- At 2×, the predicted digits lag behind. In the first predicted frame, the brightness moves only about a third of the true displacement (1.4 px against 4.6 px).
- At 3×, the predictions don't follow the digits at all. They stay near where the digits were in the last input frame and smear out.

A digit that stalls and blurs suggests the model is applying the slow motion rule it learned and hedging over the plausible positions near that slow guess. It is not tracking the fast digit.

Q2.10

No. Two explanations predict the same failure:

- Unfamiliar motion: model A never saw motion above about 5.7 px/frame.
- Reach: 3× exceeds its 8 px reach.

Both say A should fail at 3×, and A failing at 2× even though 2× is within reach hints at unfamiliarity, but with only model A the two causes can't be separated. The experiment is to vary one factor at a time:

1. Train the same architecture with speed augmentation, i.e. 1×, 2× and 3× clips. This is model C.
2. If C fixes both 2× and 3×, the failure was unfamiliar motion.
3. If C fixes 2× but still fails at 3×, the 3× failure is reach.
4. Confirm by keeping the speed-augmented data and enlarging only the reach (model D). D should then succeed at 3× where C did not.

4. What Can Data Fix?

Q2.11

Prediction committed before training models B–D (f6fecaa, 29 Oct 2026 18:59), reproduced unchanged:

Model B (symmetry augmentation) should shrink model A's mirror and rotation gaps, with C_R falling more than C_M because rotated digits were the less familiar appearance; it should leave the speed failure unchanged. Model C (speed augmentation) should fix most of the 2× degradation, since 2× motion (≈6.6 px/frame) is within the 8 px reach and was only unfamiliar, but it should still fail at 3× (≈10 px/frame), which exceeds the reach. Model D (larger reach, also speed-augmented) should be the only model that does well at 3×, which would show the 3× failure is architectural reach, not missing data.

Table 5. Models A–D. Inference: wall-clock time for the 1,000 test videos at batch size 32. Training: mean wall-clock time per epoch over the 30 epochs. Learned models: mean (seed 0, seed 1).

Model	$E^{(5)}$	C_M	E_M	C_R	E_R	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Inference (s)	Params	Train (s/epoch)
A	2.18e-2 (2.23e-2, 2.13e-2)	1.86e-3 (1.73e-3, 1.99e-3)	2.25e-2 (2.29e-2, 2.20e-2)	3.04e-3 (2.66e-3, 3.43e-3)	2.35e-2 (2.38e-2, 2.32e-2)	1.84e-2 (1.89e-2, 1.78e-2)	4.45e-2 (4.33e-2, 4.57e-2)	5.14e-2 (5.05e-2, 5.22e-2)	0.231 (0.234, 0.229)	480,784	12.5 s (16.4, 8.5)
B	2.24e-2 (2.26e-2, 2.21e-2)	1.28e-3 (1.35e-3, 1.21e-3)	2.24e-2 (2.27e-2, 2.21e-2)	1.60e-3 (1.62e-3, 1.59e-3)	2.24e-2 (2.27e-2, 2.21e-2)	1.86e-2 (1.88e-2, 1.85e-2)	4.47e-2 (4.41e-2, 4.53e-2)	5.10e-2 (4.82e-2, 5.38e-2)	0.230 (0.230, 0.230)	480,784	9.1 s (6.7, 11.5)
C	2.72e-2 (2.72e-2, 2.73e-2)	1.13e-3 (1.36e-3, 8.92e-4)	2.74e-2 (2.73e-2, 2.75e-2)	2.43e-3 (3.17e-3, 1.69e-3)	2.83e-2 (2.84e-2, 2.83e-2)	2.37e-2 (2.36e-2, 2.37e-2)	2.88e-2 (2.89e-2, 2.88e-2)	3.34e-2 (3.33e-2, 3.35e-2)	0.226 (0.227, 0.225)	480,784	14.8 s (18.6, 11.0)
D	2.87e-2 (2.85e-2, 2.88e-2)	1.08e-3 (1.19e-3, 9.72e-4)	2.89e-2 (2.88e-2, 2.91e-2)	1.85e-3 (1.83e-3, 1.86e-3)	2.97e-2 (2.95e-2, 2.99e-2)	2.64e-2 (2.63e-2, 2.65e-2)	2.92e-2 (2.91e-2, 2.94e-2)	3.17e-2 (3.17e-2, 3.16e-2)	0.244 (0.242, 0.246)	480,784	6.9 s (6.7, 7.2)

Model D was chosen on the validation split only (videos 8000–8999): dilation 2 had the lowest validation $E^{(2)}$ at 3× (3.12e-2) among the candidates, against 3.28e-2 for C; dilation 3 and patch 8 stayed near a blank prediction (3.74e-2, 3.95e-2). Full comparison: `results/model_d_selection.md`.

Q2.12

Model D is model A with a dilation of 2 in the 3×3 convolutions of both ConvLSTM layers. The padding is 2, so the grid stays at 16×16. Everything else matches model A: patch size 4, 64 hidden channels, the 1×1 output, the 9-step rollout, and the same training protocol with speed augmentation as C. Dilation doesn't add any weights, so D has 480,784 parameters. That is the same as A and well below the 721,176 cap.

A dilated 3×3 kernel connects cells up to 2 apart (8 px) instead of 1. Following the same paths as in Q2.3:

- (i) from u_τ through layer 1 (2 cells) and then layer 2 (2 cells): 4 cells, or 16 px;
- (ii) from $h^{(1)}_{\tau-1}$ through the same two convolutions: also 16 px;
- (iii) from $h^{(2)}_{\tau-1}$ through layer 2 only: 8 px.

D's reach is therefore 16 px per step. That is strictly more than A's 8 px and above the roughly 10 px/frame motion at 3×. I picked D using the validation split (`results/model_d_selection.md`):

- **Dilation 2:** lowest validation $E^{(2)}$ at 3×, 3.12e-2, compared with 3.28e-2 for C.
- **Dilation 3** (24 px reach) and **patch 8** (16 px reach, 594,496 parameters): both stayed at the error level of a near-blank prediction for most of the 30 epochs.
- **5×5 kernel:** 1,332,752 parameters, which is over the cap.

This design has a cost. A dilation-2 kernel only connects cells two apart, so the folded grid splits into four separate 8×8 sub-grids (odd/even rows × odd/even columns). These never share information. I confirmed this directly. Each output cell of D depends on 64 input cells, all in its own sub-grid. Each output cell of A depends on 169 input cells spread across all four (`results/answer_stats.json`, "coupling"). D gains reach by giving up spatial resolution.

Q2.13

- **Mirror and rotation failure: appearance.** Model B has the same motion as A but sees every orientation during training, and the penalty disappears. Its E_M and E_R both equal its $E^{(5)}$ (2.24e-2 each), whereas A had +3% and +8%. C_R drops from 3.04e-3 to 1.60e-3 (−47%) and C_M from 1.86e-3 to 1.28e-3 (−31%). Speed augmentation on its own doesn't remove the rotation penalty, since C's E_R is still 4% above its $E^{(5)}$. Part of the lower C_M for C and D comes from blurrier predictions (Q2.6). For those models, $E_T - E^{(5)}$ is a better comparison than C_T.
- **2× failure: unfamiliar motion.** C uses the same architecture with speed augmentation, and it reduces $E^{(2)}$ at 2× from 4.45e-2 to 2.88e-2 (−35%). That is well below Blank's 4.26e-2. B, which never saw fast clips, doesn't help (4.47e-2).
- **3× failure: mostly unfamiliar motion, with a small reach effect.** C alone reduces $E^{(2)}$ at 3× from 5.14e-2 to 3.34e-2 (−35%). Doubling the reach in D only brings it down to 3.17e-2 (a further −5%), although the gap holds across seeds (C 3.33e-2 / 3.35e-2, D 3.17e-2 / 3.16e-2). C probably copes at 3× despite its 8 px reach because a digit is about 20 px wide. After a 10 px jump, part of the digit is still within 8 px of where it was.

The clean result the question describes (C fails at 3×, D succeeds) didn't happen here. If it had, the 3× failure would be architectural. C and D see the same data and differ only in reach, and more data can't fix a reach limit. In a single step, each cell's update only sees cells within its reach. If a digit jumps further than that, the cell at its new position gets no information from its old position, and no choice of weights can connect the two. Data sets the weights, but it doesn't change which cells are connected.

Q2.14

Yes, speed augmentation costs accuracy at 1×:

- $E^{(5)}$: 2.18e-2 for A, 2.72e-2 for C (+25%), 2.87e-2 for D (+32%).
- $E^{(2)}$ at 1×: 1.84e-2, 2.37e-2 and 2.64e-2.

A likely reason is that the same capacity now has to cover three speeds. Only a third of training clips are at 1×, and the model has to work out the speed from five frames. D loses a further 11% at 1× compared with C, probably because of its four decoupled sub-grids (Q2.12). Symmetry augmentation, by contrast, costs very little: B's $E^{(5)}$ is 2.24e-2, only +3%.

Whether the trade-off is worth it depends on the inputs:

- **If clips can arrive at any speed,** speed augmentation clearly wins. Averaged over 1×, 2× and 3×, $E^{(2)}$ is 3.81e-2 for A, 2.86e-2 for C and 2.91e-2 for D.
- **If only 1× clips matter,** A is still best.

D's extra reach gained 5% at 3× but lost 11% at 1×, so on these numbers it isn't worth choosing over C. B's consistency gain is worth its 3% cost.

5. The Final Model

Table 6. Symmetrised models (average over all eight transformations). The training-time column is that of the underlying learned model; symmetrisation needs no extra training. The final model is symmetrised D. Mean (seed 0, seed 1).

Model	$E^{(5)}$	C_M	E_M	C_R	E_R	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Inference (s)	Params	Train (s/epoch)
sym. B	2.17e-2 (2.20e-2, 2.15e-2)	5.25e-17 (5.17e-17, 5.32e-17)	2.17e-2 (2.20e-2, 2.15e-2)	5.37e-17 (5.31e-17, 5.43e-17)	2.17e-2 (2.20e-2, 2.15e-2)	1.82e-2 (1.83e-2, 1.81e-2)	4.41e-2 (4.35e-2, 4.47e-2)	5.01e-2 (4.75e-2, 5.26e-2)	2.00 (2.01, 2.00)	480,784	9.1 s (6.7, 11.5)
sym. D	2.86e-2 (2.84e-2, 2.88e-2)	3.05e-17 (3.16e-17, 2.94e-17)	2.86e-2 (2.84e-2, 2.88e-2)	3.15e-17 (3.26e-17, 3.05e-17)	2.86e-2 (2.84e-2, 2.88e-2)	2.64e-2 (2.62e-2, 2.66e-2)	2.91e-2 (2.89e-2, 2.93e-2)	3.14e-2 (3.14e-2, 3.14e-2)	1.98 (2.00, 1.95)	480,784	6.9 s (6.7, 7.2)
sym. DB	2.86e-2 (2.86e-2, 2.86e-2)	2.85e-17 (2.92e-17, 2.79e-17)	2.86e-2 (2.86e-2, 2.86e-2)	2.93e-17 (3.00e-17, 2.86e-17)	2.86e-2 (2.86e-2, 2.86e-2)	2.64e-2 (2.64e-2, 2.63e-2)	2.93e-2 (2.92e-2, 2.93e-2)	3.15e-2 (3.15e-2, 3.14e-2)	2.00 (2.03, 1.97)	480,784	8.8 s (10.2, 7.5)

Final model: **symmetrised D**. D+B was also trained and wrapped, but Table 6 shows no gain over symmetrised D ($E^{(5)}$ 2.86e-2 for both; $E^{(2)}$ at 3× 3.15e-2 against 3.14e-2), so the simpler model D was kept.

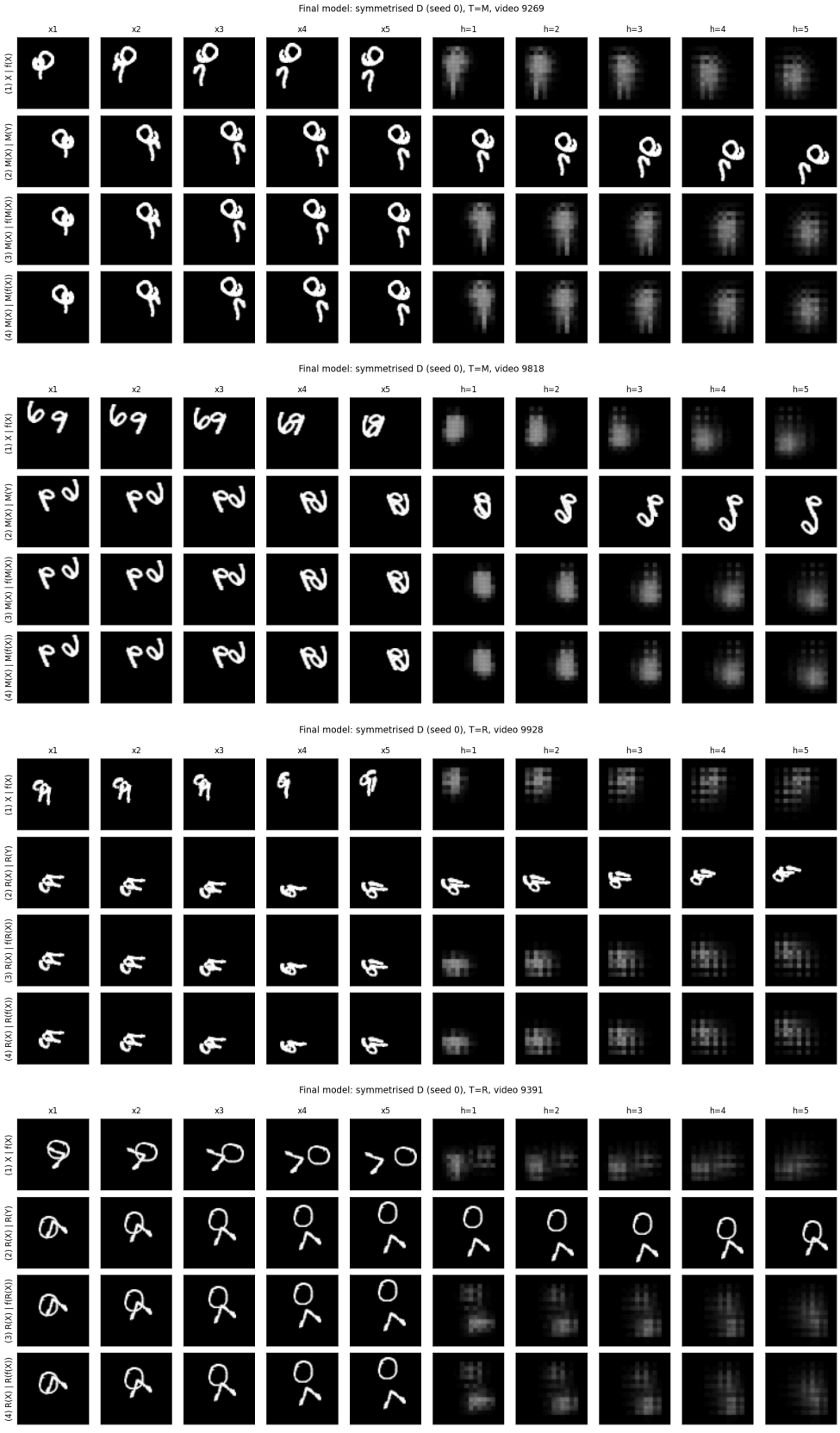


Figure 6(a). Figure 3 repeated with the final model (symmetrised D, seed 0) on the same four videos and transformations, same row layout. GIFs: `figures/gifs/fig6a_M1_v9269.gif`, `figures/gifs/fig6a_M2_v9818.gif`, `figures/gifs/fig6a_R1_v9928.gif`, `figures/gifs/fig6a_R2_v9391.gif`.

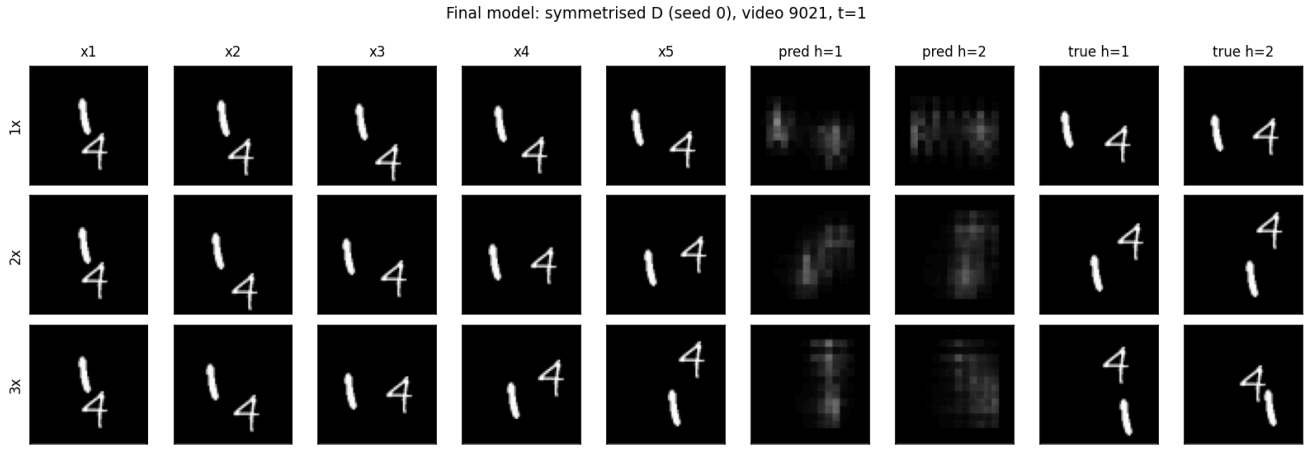


Figure 6(b). Figure 5 repeated with the final model (symmetrised D, seed 0) on test video 9021.

Q2.15

- **Exactness.** Built-in symmetry is exact, but learned symmetry isn't. Take $\bar{f}(gX) = (1/8)\,\Sigma_{h\in G}\, h^{-1} f(hgX)$. Let $k = hg$. As h runs over G, so does k, and $h^{-1} = g\,k^{-1}$. Each term becomes $g\,k^{-1} f(kX)$. g only rearranges pixels, so it can be moved outside the average: $\bar{f}(gX) = g \cdot (1/8)\,\Sigma_{k\in G}\, k^{-1} f(kX) = g\,\bar{f}(X)$. This means $C_M = C_R = 0$ for any forecaster. Table 6 confirms it up to floating-point error: about 5e-17 for symmetrised B and 3e-17 for symmetrised D. Model B learned the symmetry from augmented data, and it only brings the gaps down to C_M = 1.28e-3 and C_R = 1.60e-3. Augmentation encourages consistency but doesn't guarantee it.
- **Accuracy.** Wrapping didn't cost any accuracy. Symmetrised B's $E^{(5)}$ is 2.17e-2, compared with 2.24e-2 for B. Since the wrapper is exactly consistent, $E_M = E_R = E^{(5)}$ = 2.17e-2. At 2× and 3×, $E^{(2)}$ is 4.41e-2 and 5.01e-2, against 4.47e-2 and 5.10e-2 for B. Symmetrised D matches D: 2.86e-2 against 2.87e-2.
- **Inference cost.** It needs eight forward passes, about 2.0 s for the 1,000 test videos compared with 0.23 s for the unwrapped models, so roughly 8.7× slower. Training cost is unchanged.
- **When averaging should hurt.** Averaging eight predictions blurs the result wherever they disagree, for example when the model does well on upright clips but badly on rotated ones. The average then combines sharp, correct predictions with wrong ones. On average this didn't hurt here (see the accuracy point above), but the blur shows up on hard videos. In Q2.16, symmetrised D's brightest pixel on video 9928 is only 0.58, compared with 0.80 for model A's prediction on the same rotated clip.

Q2.16

Only partly. In Figure 6(a), rows 3 and 4 of the final model for video 9928 under rotation are now identical, with a per-horizon C of about 3e-17. The disagreement between them is gone, and symmetrisation alone fixed it. But the shared prediction still goes wrong in the same way as model A's row 3:

- The two 9s still rise too slowly, at about 1.9 px per frame against the true 3. They end 3 px short of the truth.
- They still drift right, from column 22 to 25, while the true digits stay near column 17.
- They are dimmer from the first frame, with a brightest pixel of 0.58 falling to 0.50. This is blur from the averaging.

Its error on the rotated clip is 2.00e-2 at h = 1 and 2.99e-2 at h = 5. Model A's is 1.65e-2 and 2.91e-2, so the final model is no better on this video. Speed augmentation and reach don't come into it, because this is a 1× clip. The overlap between the two 9s probably accounts for some of the remaining failure. Symmetrisation made the model consistent, not correct.

The speed failure, on the other hand, is fixed. On the Figure 5 video (9021, Figure 6(b)), $E^{(2)}$ at 2× drops from 3.70e-2 to 2.04e-2, and at 3× from 3.92e-2 to 2.25e-2. That improvement comes from speed augmentation and reach, at a small cost at 1× (1.96e-2 against 1.70e-2).

Q2.17

Partly. The final model, symmetrised D, beats Blank and Copy-last at every speed and gets worse only slowly as clips speed up. Its $E^{(2)}$ is 2.64e-2, 2.91e-2 and 3.14e-2 at 1×, 2× and 3×, compared with 1.84e-2, 4.45e-2 and 5.14e-2 for model A. This speed robustness is the best evidence that it learned a motion rule. A model that had memorised typical training clips would fail exactly here, as model A did. Its exact mirror and rotation consistency ($C_M = C_R = 0$) is not evidence, because symmetrisation guarantees that for any forecaster. But it is less accurate than model A on 1× clips ($E^{(5)}$ 2.86e-2 against 2.18e-2), its predictions blur over the five horizons, and on video 9928 it still slows and drifts the two overlapping 9s. One test it hasn't passed is keeping overlapping digits sharp and on their true paths for all five horizons.

Notes on the experiments

- **Predictions.** Q2.5 was committed in `ff9a9db` before the experiments in Testing the Baseline, and Q2.11 in `f6fecaa` before models B–D were trained (`predictions/`). Both are reproduced above unchanged. Before writing Q2.5 I had seen model A's 2× and 3× validation errors from the training monitor. The wording of both predictions started from an AI draft that I edited (Appendix).

- **Timing.** Epoch times on the laptop GPU varied widely for the same architecture (6.6–23.9 s per epoch), so the training-time columns compare models only roughly.
- **Re-run.** The first seed-0 run of model B had one epoch that stalled for 2,148 s. It was re-run with the same settings; the old log is kept in `logs/superseded/`.

Appendix — Generative AI usage

All generative-AI use for this assignment, in order: each prompt (verbatim), a summary of the output and the files it touched, and the edits I made. Tool: Claude Code (Claude Opus 5.5). The full log is `AI_USAGE_LOG.md` in the repository.

#	Date	Prompt (verbatim)	Output (summary + files)	My edits / verification
1	2026-10-14	"Analyze the PA2 and what are the deliverables"	Read `DL4STG_PA2.pdf` and the Question1 folder. Summarised the deadline, submission rules (PDF report, public repo link, AI appendix, committed predictions for Q2.5/Q2.11), the six required Q1 headings, and the Q2 models A–D, Tables 1–6, Figures 1–6 and Q2.1–Q2.17. Suggested a 12-day plan. No files changed.	Read for orientation only; nothing copied into the report.
2	2026-10-15	"Walk me through this so that I can implement this? where to start?"	Step-by-step implementation guide for Q2: environment, repo layout, order of work, and starter code for transforms/windows, the Table 1 checks, the Farneback flow (Figure 1), the ConvLSTM (model A, 480,784-parameter check), the masked loss/training loop and the evaluation harness. No files changed.	
3	2026-10-15	"You can use conda env ml310 for Step 0. Here is the Github repo for PA2 https://github.com/Awais65/DL4STG-PA2.git. Lets create a log book now for the prompts where we can add the prompts. Once done lets start with the next steps"	Installed `opencv-python` and `imageio` into `ml310`. Cloned the repo. Created this log, `README.md`, `.gitignore`, `requirements.txt`, `q2/data.py` (loading, transforms, windows), `q2/checks.py` (Table 1), `q2/flow.py` (Figure 1) and `q2/models.py` (ConvLSTM forecaster). Ran them: all Table 1 checks pass (`results/table1.md`), model A has 480,784 parameters, and Figure 1 was generated (`figures/fig1_flow.png`; 103,050 pixels kept, median 3.32 px/frame).	
4	2026-10-25	"check PA2 progress and whats remaining" / "yes, commit and start the training code"	Status review against the handout. Made the first git commit. Wrote `q2/train.py` (training protocol for models A, B, C, D, D+B: masked loss, speed/symmetry augmentation, seeding, epoch timing, validation monitoring); checked window sampling and timed one epoch (18.6 s on RTX 4070 Laptop). Started model A training, seeds 0 and 1.	
5	2026-10-25	"was anything missing?" / "Yes"	Audited `q2/train.py` and the model A runs against the training protocol (all requirements met). Flagged gaps: no 2x/3x validation metric for choosing model D, GPU nondeterminism, unstable epoch timings on the laptop GPU, uncommitted results. Replaced the validation monitor with `val_metrics` (val E^(5) at 1x and E^(2) at 1x/2x/3x, t=1).	
6	2026-10-25	"Give me an artificate of tasks and pending completion"	Published a private progress-tracker web page (claude.ai artifact) listing phases, tables, figures, answers and training runs with their status. No repo files changed.	
7	2026-10-25	"push the code for now"	Push failed (git credential helper set to AWS CodeCommit); gave the command to push through Git Credential Manager. No files changed.	
8	2026-10-25	"in artifacte add code locations to review and pending questions anaylsis as well"	Added to the tracker: 13 file:line locations to check against the handout, and for each of Q2.1-Q2.17 and Q1 what it asks, what it depends on and what to cite (no answers written). No repo files changed.	
9	2026-10-28	"whats pending?"	Status check: nothing pushed yet, list of remaining deliverables. No files changed.	
10	2026-10-28	"the deadline was changed start with the pending task"	Wrote `q2/evaluate.py` (per-horizon MSE, E^(5), E^(2), C_T, E_T, Blank, Copy-last, symmetrised forecaster, inference timing, Tables 2-6; Tables 3-6 refuse to run until `predictions/Q2.5.md` / `predictions/Q2.11.md` are committed). Checked on random tensors that the symmetrised forecaster gives C_M, C_R ~1e-15. Ran Table 2 (`results/table2.md`). Wrote `q2/figures.py` (Figures 2-6, GIFs; 3-6 gated the same way), generated Figure 2 (`figures/fig2_examples.png`, videos 9002, 9007, 9191 with overlap); Figures 3-6 tested only on synthetic data with an untrained model.	
11	2026-10-29	"commit checkpoints and logs too" / "start with remaining tasks will add the writeouts in the end"	Committed `checkpoints/` and `logs/`. Pointed out that Q2.5/Q2.11 must be committed before their experiments; I chose to write short predictions now.	
12	2026-10-29	"Give me predections verbaige and will modify it"	AI drafted a Q2.5 prediction (mirror passes mostly, rotation harder, breakdown at 3x) with a placeholder for my reach estimate. Noted that I had already seen validation E^(2) at 2x/3x on 2026-10-25 (1.85e-2 / 4.27e-2 / 4.99e-2), so the speed part is not fully blind.	I rewrote and shortened the draft (my text pasted back), changed the rotation example to "a sideways 1 resembles a dash".
13	2026-10-29	"reach is 8 px, correct this and remove placeholders"	Replaced "[R]" with 8 px and removed "[If R ≈ 8:]"; committed my text otherwise unchanged as `predictions/Q2.5.md` before running Tables 3-4 / Figures 3-5.	Reach of 8 px is my Q2.3 estimate.
14	2026-10-29	(continuation of 13: run the baseline tests)	Ran Table 3 (mirror/rotation), Table 4 (2x/3x), Figure 3 + GIFs (videos 9269, 9818 for M; 9928, 9391 for R), Figure 4 and Figure 5. Figure 5 first picked video 9007, whose digits touch at 2x/3x; added a connected-component check to `fig5_video` and regenerated with video 9021.	
15	2026-10-29	"Give me the verbiage and I'll modify"	AI drafted a Q2.11 prediction (B fixes mirror/rotation gaps, C fixes 2x but not 3x, D fixes 3x), written after seeing Tables 3-4 and before training B-D. The draft was 4 sentences; AI flagged this and proposed joining sentences 1-2 with a semicolon.	I sent the draft back unchanged.
16	2026-10-29	"Approved go ahead and push it"	Committed the 3-sentence version as `predictions/Q2.11.md` and pushed before any B/C/D training.	Approved the semicolon merge.
17	2026-10-29	(after Q2.11 commit)	Wrote `scripts/run_bcd.sh` (B, C seeds 0/1; model-D candidates dilation 2, dilation 3, patch 8 on seed 0) and `q2/select_d.py` (validation-only comparison). A 5x5 kernel was ruled out: 1,332,752 params > 721,176.	
18	2026-10-29	"update the tracker artifact while training runs" / "is the training running?" / "training status?"	Updated the progress tracker; reported queue progress. B seed 0 had one stalled epoch (2,148 s), so its mean epoch time (80.4 s) is not representative.	
19	2026-10-29	(queue finished)	Validation comparison (`results/model_d_selection.md`): dilation 2 lowest val E^(2) at 3x (3.12e-2 vs C 3.28e-2); dilation 3 and patch 8 stuck on a near-blank plateau (loss ~4.02e-2) for most of the 30 epochs. Noted epoch-time variance on the laptop (6.6-23.9 s for the same architecture).	
20	2026-10-29	"go with dilation 2 and train D"	Training model D = dilation-2 ConvLSTM (480,784 params), seeds 0 and 1.	I chose dilation 2 from the validation comparison.
21	2026-10-29	(continuation of 20)	Ran Table 5 (A-D), Table 6 (symmetrised B and D; C_M, C_R ~3e-17 to 5e-17) and Figure 6 (symmetrised D seed 0 on the Figure 3 and Figure 5 videos, GIFs `figures/gifs/fig6a_*.gif`).	
22	2026-10-29	"re-run B seed 0 and update the tracker"	Re-trained B seed 0 (mean epoch 6.7 s; the stalled run is kept in `logs/superseded/`), re-ran Tables 5 and 6, updated the tracker.	

#	Date	Prompt (verbatim)	Output (summary + files)	My edits / verification
23	2026-10-29	"train D+B too"	Trained D+B (dilation 2, speed + symmetry augmentation), seeds 0 and 1; re-ran Table 6 with symmetrised D+B as a third row. Figure 6 still uses symmetrised D.	
24	2026-10-29	"keep symmetrised D as final"	No change: Figure 6 already uses symmetrised D (seed 0).	I chose symmetrised D as the final model.
25	2026-10-29	"give me verbiage for Q2.1 to Q2.4"	AI drafted answers to Q2.1-Q2.4 from Figure 1, Table 2 and Figure 2, plus extra Figure 1 statistics (total variation distance of the direction histogram under mirror/rotation; speed percentiles).	
26	2026-10-29	(pasted the Q2.1-Q2.4 draft back)	Saved it as `report/answers_q2.md`. While adding the extra statistics to `q2/flow.py` / `results/fig1_flow_stats.json`, AI found its draft was wrong: 21 of 103,050 kept pixels (max 8.1 px/frame) move faster than 6.6 px/frame, not none. Corrected the Q2.1 sentence.	Pasted the draft back without changes.
27	2026-10-30	"give me verbiage for Q2.5 to Q2.10" / "whats pending?" / "yes"	AI measured the Figure 3/5 panels (brightness centroids, peak, bright area), Blank's E^(2) per speed and per-video C_M vs C_R, then drafted Q2.6-Q2.10 and an optional labelled outcome note for Q2.5 (Q2.5 itself kept verbatim).	
28	2026-10-30	(pasted the Q2.5-Q2.10 draft back)	Appended to `report/answers_q2.md` with the committed Q2.5 text inserted unchanged. Added `q2/answer_stats.py -> results/answer_stats.json` so the cited numbers are reproducible. AI corrected one wording of its own draft in Q2.7 (row-4 drift is about 1 px, not under 1 px).	Pasted the draft back without changes.
29	2026-10-30	"figure 9928 have 9 and 1?" / "update Q2.7 to say two overlapping 9s"	AI rendered all 20 frames of test video 9928: both digits appear to be 9s and overlap throughout. Updated Q2.7 in `report/answers_q2.md` to name them.	I identified the digits as two 9s from the frames.
30	2026-10-30	"give me drafts for the remaining questions"	AI measured the final model on the Figure 3/5 videos and the cell coupling of models A and D (model D splits into four 8x8 sub-grids), added both to `q2/answer_stats.py`, then drafted Q2.12-Q2.17 and an optional outcome note for Q2.11.	
31	2026-10-30	(pasted a reworded Q2.12-Q2.17 with six bracketed gaps)	Saved to `report/answers_q2.md` with the committed Q2.11 text inserted unchanged. AI filled the gaps from the results (symmetrised B C ~5e-17; 1,000 test videos; 0.23 s unwrapped; truth near column 17; 1x cost 1.96e-2 vs 1.70e-2; Q2.17 'beats Blank and Copy-last at every speed'). Q2.17 came to 153 words, so AI removed 'A sceptic should also weigh the costs.' to reach 147.	I reworded the Q2.12-Q2.17 draft.
32	2026-10-30	"Draft them" (the six Q1 sections)	AI read the Question 1 companion notebook for course vocabulary, then drafted all six Q1 sections in `report/answers_q1_draft.md`: a directed river graph (reservoir, stations, latent unmonitored-tributary node) with causal temporal blocks, flow-dependent lag-attention routing and a quantile head; assumptions A1-A10; input/output table; equations; missing/delayed data and release handling; limitations and information requests. Not committed until reviewed.	
33	2026-10-31	"yes start assembling the report" / "Name is Muhammad Awais Ismail Roll Number: 24030007"	AI wrote `report/build_report.py`, which assembles `report/PA2_report.pdf` (via `PA2_report.html`, printed with headless Chrome) from the answer files, `results/table*.md` (rounded to 3 significant figures), the figures, and this log as the appendix. Page 1 has name, roll number, repo link and hardware; Q2 follows the handout sections with each table and figure before its questions. AI wrote the figure/table captions, the final-model note (sym. D kept: sym. D+B no better in Table 6) and the "Notes on the experiments" disclosures. Wrapped `X_rel_future` in backticks in the Q1 draft (formatting only).	
34	2026-10-31	"correct it remove any placeholders the version is fine"	AI removed the draft notes and moved Q1 to `report/answers_q1.md`. Corrections: rain totals (`X_rainctx`) added to the input-embedding equation; calendar input added to the diagram; S1 no longer said to use R1 rain (A6 maps R1 to T1/S2); monotone response (A5) stated as a verification check, not built into the network; q, k_l and lambda defined; attention cost O(	E